

```
"""
```

Project 2: The Expense Tracker

Goal:

Create a script where users enter expense amounts. The program adds them up and displays the Total Spent.

Key Skill:

Math operations & Accumulators ($\text{total} = \text{total} + \text{new_expense}$)

```
"""
```

```
QUIT_COMMAND = "quit"
```

```
def show_intro():
```

```
    """OUTPUT: explain how the program works before it starts."""
```

```
    print("\n" + "-" * 36)
```

```
    print("    EXPENSE TRACKER")
```

```
    print("-" * 36)
```

```
    print("Enter an expense amount each time you're asked.")
```

```
    print(f"Type '{QUIT_COMMAND}' when you're done to see your total.\n")
```

```
def read_expense():
```

```
    """
```

INPUT: ask for one entry and return the raw text exactly as typed. Converting it to a number happens later - this function only collects what the user typed.

```
    """
```

```
    return input("Expense amount (or 'quit' to finish): ").strip()
```

```
def main():
```

```
"""
```

Entry point.

total is the accumulator this whole program is built around.

It MUST be set here, outside the loop, so it's created once.

If this line were inside the loop instead, every new entry

would wipe it back to 0 before adding anything - the

"Iteration Amnesia" problem the training material warns about.

```
"""
```

```
total = 0 # State initialized OUTSIDE the loop
```

```
show_intro()
```

```
while True: # Runs until the kill switch breaks it
```

```
    entry = read_expense()
```

```
    # Kill switch: a sentinel value that ends the loop on
```

```
    # command, so the program doesn't run forever.
```

```
    if entry.lower() == QUIT_COMMAND:
```

```
        break
```

```
    # Defensive coding: input() always returns a string, so
```

```
    # '100' + '50' would give '10050', not 150. int() converts
```

```
    # it to a real number, and a non-numeric entry raises
```

```
    # ValueError instead of crashing the whole program.
```

```
    try:
```

```
        expense = int(entry)
```

```
    except ValueError:
```

```
        print("Invalid input - please enter a whole number, or 'quit'.\n")
```

```
        continue
```

```
    # Accumulator pattern: new state = old state + input.
```

```
total += expense
```

```
print(f"Added {expense}. Running total: {total}\n")
```

```
# Output phase: the final result, shown once everything above is done.
```

```
print(f"FINAL TOTAL: ${total:.2f}")
```

```
if __name__ == "__main__":
```

```
    main()
```